

算法原理 HW4 作业

姓名：冯峻 学号：523031910148

2025 年 11 月 7 日

目录

1. 题目：给定正整数集合 $S = \{a_1, a_2, \dots, a_n\}$ ，设计动态规划算法求解该集合的划分问题，即判断是否存在集合 $A \subseteq S$ ，使得 $\sum_{a_i \in A} a_i = \sum_{a_i \in S-A} a_i$ 。分析该问题的优化子结构、重叠子问题，给出动态规划算法伪代码并分析算法的复杂度。

解：

设计过程：

该问题等价于判断是否存在子集 A ，使得其元素之和等于集合总和的一半。设集合 S 的总和为 sum ，若 sum 为奇数，则显然无解，因为划分后的两组数之和相同的话，这必然意味着原数组总和一定是偶数，因为原数组中的每个数都是正整数；若 sum 为偶数，这才是我们需要考虑的问题，这样的话问题就直接转化为：是否存在子集，其元素之和等于 $\text{target} = \text{sum}/2$ 。

优化子结构：

定义子问题： $dp[i][j]$ 表示使用前 i 个元素能否凑出和为 j 。对于第 i 个元素 a_i ，有两种选择：

- 不选 a_i ：问题变为前 $i-1$ 个元素能否凑出和为 j ，即 $dp[i][j] = dp[i-1][j]$
- 选 a_i ：问题变为前 $i-1$ 个元素能否凑出和为 $j - a_i$ ，即 $dp[i][j] = dp[i-1][j - a_i]$

因此，原问题的最优解包含子问题的最优解，具有**优化子结构**性质。

重叠子问题：

在计算 $dp[i][j]$ 时，可能需要多次计算相同的子问题 $dp[i-1][j']$ 。例如，计算 $dp[3][10]$ 和 $dp[3][12]$ 时，都可能需要 $dp[2][8]$ 的结果。这种重复计算的子问题体现了**重叠子问题**性质。

动态规划算法伪代码：

Algorithm SUBSET-PARTITION(S)

```

1:  $\text{sum} \leftarrow \sum_{i=1}^n S[i]$ 
2: if  $\text{sum} \bmod 2 \neq 0$  then
3:   return False
4:  $\text{target} \leftarrow \text{sum}/2$ 
5: 初始化二维数组  $dp[0..n][0..\text{target}]$ , 全部置为 False
6:  $dp[i][0] \leftarrow \text{True}, \forall i \in [0, n]$  ▷ 和为 0 总是可以达到
7: for  $i = 1$  to  $n$  do
8:   for  $j = 1$  to  $\text{target}$  do
9:      $dp[i][j] \leftarrow dp[i-1][j]$  ▷ 不选第  $i$  个元素
10:    if  $j \geq S[i]$  then
11:       $dp[i][j] \leftarrow dp[i][j] \vee dp[i-1][j-S[i]]$  ▷ 选第  $i$  个元素
12: return  $dp[n][\text{target}]$ 

```

时间复杂度分析:

算法需要填充一个 $n \times \text{sum}/2$ 的动态规划表。每个状态的计算时间为 $O(1)$ 。因此, 总时间复杂度为 $O(n \cdot \text{sum}/2)$ 。

空间复杂度分析:

需要维护一个维度为 $n \times \text{sum}/2$ 的动态规划表, 因此空间复杂度为 $O(n \cdot \text{sum}/2)$ 。

2. 题目：课上讲过了子串和子序列的区别。现考虑两个字符串 X 和 Y ，求解其最长的公共子串。比如 $X = \text{'photograph'}$ ， $Y = \text{'tomography'}$ ，则其最长公共子串为 'ograph' 。

1) 已知 $n = |X|$ ， $m = |Y|$ ，请设计一个时间复杂度 $\Theta(nm)$ 的动态规划算法求解 X 和 Y 的最长公共子串。

2) 设计一个时间复杂度 $\Theta(nm)$ 的非动态规划算法求解 X 和 Y 的最长公共子串。

需给出相应的算法设计和分析过程，不能仅仅只给出伪代码。

解：

1) 动态规划算法

优化子结构分析：

定义状态 $dp[i][j]$ 表示以 $X[i-1]$ 和 $Y[j-1]$ 结尾的最长公共子串的长度。

状态转移方程：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & \text{if } X[i-1] = Y[j-1] \\ 0, & \text{if } X[i-1] \neq Y[j-1] \end{cases}$$

当 $X[i-1] = Y[j-1]$ 时，以它们结尾的最长公共子串长度等于以 $X[i-2]$ 和 $Y[j-2]$ 结尾的最长公共子串长度加 1。当字符不相等时，由于子串必须连续，长度为 0。

这与最长公共子序列不同。在子序列问题中，当字符不相等时，状态转移为 $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ ，因为我们没有当前位置的字符串匹配失败了，不代表当前记录的信息无法再使用，可能可以在后续匹配时用到。但是如果我们考虑的是连续的字符串，情况就有所不同了，由于连续性要求，我们一旦发现当前字符不再匹配，就只能立刻终止匹配，记录的信息也无法被后面利用。

重叠子问题：

在计算 $dp[i][j]$ 时，需要用到 $dp[i-1][j-1]$ 的结果。多个状态的计算会重复使用相同的子问题结果，体现了重叠子问题性质。

动态规划算法伪代码：

Algorithm LCS-DP(X, Y)

```

1:  $n \leftarrow |X|, m \leftarrow |Y|$ 
2: 初始化二维数组  $dp[0..n][0..m]$ , 全部置为 0
3:  $maxLength \leftarrow 0, endIndex \leftarrow 0$ 
4: for  $i = 1$  to  $n$  do
5:   for  $j = 1$  to  $m$  do
6:     if  $X[i - 1] = Y[j - 1]$  then
7:        $dp[i][j] \leftarrow dp[i - 1][j - 1] + 1$ 
8:       if  $dp[i][j] > maxLength$  then
9:          $maxLength \leftarrow dp[i][j]$ 
10:         $endIndex \leftarrow i - 1$ 
11:     else
12:        $dp[i][j] \leftarrow 0$ 
13: 从  $X[endIndex - maxLength + 1..endIndex]$  提取最长公共子串
14: return 最长公共子串及其长度  $maxLength$ 

```

时间复杂度：需要填充 $n \times m$ 的动态规划表，每个状态计算为 $O(1)$ ，因此总时间复杂度为 $\Theta(nm)$ 。

空间复杂度：需要维护一个维度为 $n \times m$ 的动态规划表，因此空间复杂度为 $\Theta(nm)$ 。

2) 非动态规划算法

设计过程：

枚举所有有序点对 $(i, j) (i \in [0, n - 1], j \in [0, m - 1])$ ，开始向后匹配，直到遇到不相等的字符为止。记录匹配的最大长度及其结束位置。这种方法其实就是枚举，会考虑全部可能的子串起点组合，因此一定能够找到所有公共子串，进而可以确定出最长的公共子串。

时间复杂度分析：

枚举所有有序点对 $(i, j) (i \in [0, n - 1], j \in [0, m - 1])$ ，开始向后匹配，这是一个二重循环，时间复杂度为 $O(nm)$ 。每一次操作内部同样会在进行一次循环（向后匹配）。在最坏情况下，比如两个完全相同的字符串，每次匹配都需要进行 $\min(n - i, m - j)$ 次比较，因此时间复杂度为: $O(n^3)$ 或者说 $O(mn \min(m, n))$

但这只是极端情况，对于实际过程中，大部分情况下可以认为这种向后匹配的操作进行不了几次就会终止，因此我们可以认为向后匹配的操作次数是常数时间的，因此我们可以近似认为该算法的时间复杂度是 $O(nm)$ 。

空间复杂度： $O(1)$ ，只使用常数个变量，因为我们不需要像之前一样维护一个维度与输入数据长度挂钩的动态规划表。

非动态规划算法伪代码：

Algorithm LCS-NOT-DP(X, Y)

```

1:  $n \leftarrow |X|, m \leftarrow |Y|$ 
2:  $\text{maxLength} \leftarrow 0, \text{endIndex} \leftarrow 0$ 
3: for  $i = 0$  to  $n - 1$  do                                     ▷ 枚举  $X$  的起始位置
4:   for  $j = 0$  to  $m - 1$  do                                     ▷ 枚举  $Y$  的起始位置
5:      $\text{length} \leftarrow 0$ 
6:      $x \leftarrow i, y \leftarrow j$ 
7:     while  $x < n$  AND  $y < m$  AND  $X[x] = Y[y]$  do
8:        $\text{length} \leftarrow \text{length} + 1$ 
9:        $x \leftarrow x + 1, y \leftarrow y + 1$ 
10:    if  $\text{length} > \text{maxLength}$  then
11:       $\text{maxLength} \leftarrow \text{length}$ 
12:       $\text{endIndex} \leftarrow i + \text{length} - 1$ 
13: 从  $X[\text{endIndex} - \text{maxLength} + 1..\text{endIndex}]$  提取最长公共子串
14: return 最长公共子串及其长度  $\text{maxLength}$ 

```
